COMPILER CONSTRUCTION LAB

LAB MANUAL

Lab: Top-Down Parsing using Recursive Descent Parser

FIRST/FOLLOW Set Computation | Left Recursion & Left Factoring

Department of Computer Science
Undergraduate Program — Compiler Construction Lab

Table of Contents

1. Lab Objectives

Upon completing this lab, students will be able to:

- Understand the structure and operation of top-down parsers.
- Implement a recursive descent parser from a given LL(1) grammar.
- ~~Compute FIRST and FOLLOW sets manually for any context-free grammar.~~
- Identify and eliminate immediate and indirect left recursion.
- Apply left factoring to resolve common-prefix ambiguities in productions.
- Verify whether a grammar satisfies the LL(1) condition.
- ~~[Bonus] Automate FIRST/FOLLOW computation and grammar transformations.~~

2. Prerequisites

Students should be comfortable with the following before attending this lab:

- Context-Free Grammars (CFGs): terminals, non-terminals, productions, start symbol.
- Derivations, sentential forms, parse trees, and leftmost/rightmost derivations.
- Basics of procedural or object-oriented programming (C, C++, Java, or Python).
- Introductory finite automata and regular expressions.

3. Introduction to Top-Down Parsing

3.1 What is Parsing?

Parsing is the process of analyzing a token sequence (produced by the lexer) to determine its grammatical structure with respect to a formal grammar. The result is a parse tree representing the syntactic structure of the input.

3.2 Top-Down vs. Bottom-Up Parsing

Top-down parsing begins at the start symbol and attempts to derive the input string by successively expanding the leftmost non-terminal. Bottom-up parsing starts from the input tokens and reduces them toward the start symbol. This lab focuses exclusively on top-down parsing.

Property	Top-Down (LL)	Bottom-Up (LR)
Direction	Start symbol to input	Input to start symbol
Derivation	Leftmost	Rightmost (in reverse)
Common Algorithm	Recursive Descent, LL(1)	SLR, LALR, LR(1)
Grammar Class	LL(k)	LR(k)

Implementation	Simpler, hand-written	Usually generated by tools

3.3 Types of Top-Down Parsers

- **Backtracking Parsers:** Try productions one at a time; backtrack on failure. Inefficient for large grammars (exponential worst-case).
- **Predictive Parsers:** Use a lookahead token to deterministically select a production with no backtracking. Recursive descent parsers are predictive.

3.4 Limitations Requiring Grammar Transformation

Predictive top-down parsers cannot be directly constructed for grammars that contain:

- **Left Recursion:** e.g., $A \rightarrow A\ \alpha$, which leads to infinite recursion in a recursive procedure.
- **Common Prefixes:** e.g., $A \rightarrow \alpha\ \beta \mid \alpha\ \gamma$, where the parser cannot choose between alternatives without consuming more input.

These issues are resolved by applying the grammar transformations described in Sections 8 and 9.

4. Recursive Descent Parsing

4.1 Overview

A recursive descent parser is a top-down parser consisting of a set of mutually recursive procedures, one for each non-terminal of the grammar. The parser begins by invoking the procedure for the start symbol and attempts to consume the entire input string.

4.2 Structure of a Parsing Procedure

For each production $A \rightarrow X\ \ X\ \ ...\ X$, the procedure for A performs the following:

- If X_i is a non-terminal, call the procedure for X_i.
- If X_i is a terminal, call match(X_i) to verify and consume the current input token.

The current lookahead token (next unprocessed token) determines which production to apply.

4.3 General Pseudocode Template

```
global  lookahead  ← first_token()      // Current input token

procedure match(expected):
    if lookahead = expected:
        lookahead ← next_token()
    else:
        error("Expected '"+expected+"', found '"+lookahead+"'")

procedure A():                           // For non-terminal A with productions A → α
| β
    if lookahead   FIRST(α):
        /* parse α — call procedures / match terminals */
```

```
        elif lookahead   FIRST(β):
            /* parse β */
        elif ε   productions_of(A) and lookahead   FOLLOW(A):
            /* epsilon production — do nothing */
        else:
            error("Unexpected token: " + lookahead)
```

5. FIRST Sets

5.1 Definition

FIRST(α) is the set of terminals that can appear as the first symbol of any string derivable from α. If α can derive the empty string ε, then ε is also included.

Formal definition: FIRST(α) = { a T | α →* a β } { ε | α →* ε }

where T is the set of all terminal symbols and →* denotes zero or more derivation steps.

5.2 Rules for Computing FIRST(X)

Rule 1 (Terminal): If X is a terminal, then FIRST(X) = { X }.

Rule 2 (Epsilon): If X is a non-terminal and X → ε is a production, add ε to FIRST(X).

Rule 3 (Non-terminal with RHS): If X is a non-terminal and X → Y Y ... Y is a production:

- Add FIRST(Y) \ {ε} to FIRST(X).
- If ε FIRST(Y), add FIRST(Y) \ {ε} to FIRST(X).
- Continue: if ε FIRST(Y_i) for all i from 1 to j−1, add FIRST(Y) \ {ε}.
- If ε FIRST(Y_i) for ALL i = 1 to k, then add ε to FIRST(X).

5.3 Iterative Algorithm

```
    Algorithm: Compute_FIRST

    For each terminal a:    FIRST(a) ← { a }
    For each non-terminal A: FIRST(A) ← { }

    repeat
        changed ← false
        for each production X → Y  Y  ... Y :
            i ← 1
            while i ≤ k:
                old ← FIRST(X)
                FIRST(X) ← FIRST(X)   (FIRST(Y ) \ {ε})
                if FIRST(X) ≠ old: changed ← true
                if ε   FIRST(Y ): break
```

```
            i ← i + 1
        if i > k:          // All symbols can derive ε
            if ε   FIRST(X): changed ← true
            FIRST(X) ← FIRST(X)   {ε}
until not changed
```

5.4 Worked Example — Grammar G1

We will use the following classic expression grammar throughout this manual:

```
Grammar G1:
  E  →  T E'
  E' →  + T E'  |  ε
  T  →  F T'
  T' →  * F T'  |  ε
  F  →  ( E )  |  id
```

FIRST(F):

F → (E) : first symbol is '(' (terminal) → add '(' to FIRST(F).

F → id : first symbol is 'id' (terminal) → add 'id' to FIRST(F).

Result: FIRST(F) = { (, id }

FIRST(T'):

T' → * F T' : first symbol is '*' → add '*' to FIRST(T').

T' → ε : epsilon production → add ε to FIRST(T').

Result: FIRST(T') = { *, ε }

FIRST(T):

T → F T' : add FIRST(F) \ {ε} = { (, id } to FIRST(T).

Since ε FIRST(F), we stop — ε does not propagate.

Result: FIRST(T) = { (, id }

FIRST(E'):

E' → + T E' : first symbol is '+' → add '+' to FIRST(E').

E' → ε : epsilon production → add ε to FIRST(E').

Result: FIRST(E') = { +, ε }

FIRST(E):

E → T E' : add FIRST(T) \ {ε} = { (, id } to FIRST(E).

Since ε FIRST(T), stop.

Result: FIRST(E) = { (, id }

Summary — FIRST Sets for Grammar G1:

Non-Terminal	FIRST Set
E	{ (, id }
E'	{ +, ε }
T	{ (, id }
T'	{ *, ε }
F	{ (, id }

6. FOLLOW Sets

6.1 Definition

FOLLOW(A) is the set of terminals (including the end-of-input marker $) that can appear immediately to the right of non-terminal A in some sentential form derived from the start symbol.

Formal definition: FOLLOW(A) = { a T {$} | S →* α A a β, for some strings α, β }

Important: ε is NEVER a member of any FOLLOW set.

6.2 Rules for Computing FOLLOW(A)

Rule 1 (Start Symbol): Add $ to FOLLOW(S), where S is the grammar start symbol.

Rule 2 (Middle of RHS): If there is a production A → α B β:

- Add FIRST(β) \ {ε} to FOLLOW(B).
- If ε FIRST(β), also add FOLLOW(A) to FOLLOW(B).

Rule 3 (End of RHS): If there is a production A → α B (B at the end of RHS):

- Add FOLLOW(A) to FOLLOW(B).

(This is equivalent to Rule 2 when β = ε, since FIRST(ε) = {ε}.)

6.3 Iterative Algorithm

```
Algorithm: Compute_FOLLOW
(Requires FIRST sets to be pre-computed)

FOLLOW(S) ← { $ }           // S = start symbol
For all other A: FOLLOW(A) ← { }

repeat
    changed ← false
    for each production A → X  X  ... X :
        trailer ← FOLLOW(A)
        for i ← n downto 1:
            if X  is a non-terminal:
                old ← FOLLOW(X )
                FOLLOW(X ) ← FOLLOW(X )   trailer
                if FOLLOW(X ) ≠ old: changed ← true
                if ε   FIRST(X ):
                    trailer ← trailer   (FIRST(X ) \ {ε})
                else:
                    trailer ← FIRST(X )
            else:            // X  is a terminal
                trailer ← { X  }
until not changed
```

6.4 Worked Example — Grammar G1

Using FIRST sets computed in Section 5.4:

FOLLOW(E):

E is the start symbol → add $ to FOLLOW(E).

From F → (E) : B = E, β = ')' → add FIRST(")") = {) } to FOLLOW(E).

Result: FOLLOW(E) = {), $ }

FOLLOW(E'):

From E → T E' : E' is at end of RHS → add FOLLOW(E) to FOLLOW(E').

Result: FOLLOW(E') = {), $ }

FOLLOW(T):

From E → T E' : β = E', add FIRST(E') \ {ε} = { + } to FOLLOW(T).

Since ε FIRST(E'), also add FOLLOW(E) = {), $ } to FOLLOW(T).

Result: FOLLOW(T) = { +,), $ }

FOLLOW(T'):

From T → F T' : T' is at end of RHS → add FOLLOW(T) to FOLLOW(T').

Result: FOLLOW(T') = { +,), $ }

FOLLOW(F):

From T → F T' : β = T', add FIRST(T') \ {ε} = { * } to FOLLOW(F).

Since ε FIRST(T'), add FOLLOW(T) = { +,), $ } to FOLLOW(F).

Result: FOLLOW(F) = { *, +,), $ }

Summary — FOLLOW Sets for Grammar G1:

Non-Terminal	FOLLOW Set
E	{), $ }
E'	{), $ }
T	{ +,), $ }
T'	{ +,), $ }
F	{ *, +,), $ }

7. LL(1) Grammars

7.1 Definition

A grammar G is LL(1) — read "Left-to-right scan, Leftmost derivation, 1 token of lookahead" — if and only if, for any two distinct productions A → α and A → β of the same non-terminal A, both of the following hold:

Condition 1: FIRST(α) ∩ FIRST(β) =

No terminal can begin strings derived from both α and β.

Condition 2: If ε FIRST(α), then FIRST(β) ∩ FOLLOW(A) = , and vice versa.

If one alternative can derive ε, the other must not share any terminal with FOLLOW(A).

7.2 Constructing the Predictive Parsing Table

For each production A → α in the grammar, perform the following:

- For each terminal a FIRST(α) \ {ε}, add A → α to M[A, a].
- If ε FIRST(α), for each b FOLLOW(A), add A → α to M[A, b] (including b = $).

The grammar is LL(1) if and only if no table entry M[A, a] contains more than one production.

7.3 Predictive Parsing Table — Grammar G1

Non-Terminal	**id**	**+**	*****	**(**	**)**	**$**
E	E → T E'			E → T E'		
E'		E'→+TE'			E'→ε	E'→ε
T	T → F T'			T → F T'		
T'		T'→ε	T'→*FT'		T'→ε	T'→ε
F	F → id			F → (E)		

Each cell contains at most one production — Grammar G1 is confirmed LL(1).

8. Left Recursion Removal

8.1 Definition

A grammar is **left-recursive** if it contains a non-terminal A such that A Aα for some string α (A derives a string that starts with itself, in one or more steps).

8.2 Types of Left Recursion

8.2.1 Immediate Left Recursion

A production of the form A → A α is directly left-recursive.

Example:

```
E → E + T  |  T
```

8.2.2 Indirect Left Recursion

Left recursion that occurs through a chain of two or more productions.

Example:

```
A → B a
B → A b  |  c
// Cycle: A → B a → A b a
```

8.3 Removing Immediate Left Recursion

General Pattern:

```
A → A α   |  A α   |  ...  |  A α   |  β   |  β   |  ...  |  β
// where no β  begins with A
```

Replace with:

```
A  →  β  A'  |  β  A'  |  ...  |  β  A'
A' →  α  A'  |  α  A'  |  ...  |  α  A'  |  ε
```

8.4 Worked Example 1 — Expression Grammar

Original Grammar:

```
E → E + T  |  T
T → T * F  |  F
F → ( E )  |  id
```

Step 1 — Remove immediate left recursion from E:

α = "+ T", β = "T"

```
E  → T E'
E' → + T E'  |  ε
```

Step 2 — Remove immediate left recursion from T:

α = "* F", β = "F"

```
T  → F T'
T' → * F T'  |  ε
```

Final Grammar (= Grammar G1):

```
E  → T E'
E' → + T E'  |  ε
T  → F T'
T' → * F T'  |  ε
F  → ( E )  |  id
```

8.5 General Algorithm — Handles Indirect Left Recursion

```
Algorithm: Remove_All_Left_Recursion

Input:  Grammar G with non-terminals A , A , ..., A  (any fixed ordering)
Output: Equivalent grammar with no left recursion
```

```
for i ← 1 to n:
    for j ← 1 to i-1:
        // Substitute current A  productions into A  → A  γ
        for each production A  → A  γ:
            remove  A  → A  γ
            for each production A  → δ:
                add  A  → δ γ
    // Remove any immediate left recursion now present in A
    apply immediate-left-recursion removal to A
```

8.6 Worked Example 2 — Indirect Left Recursion

Original Grammar:

```
A → B a  |  a
B → A b  |  b
```

Order: A = A, A = B

Iteration i=1 (A):

No j < 1 — inner loop skipped.

No immediate left recursion in A (A → B a | a).

Iteration i=2 (B), j=1 substituting A:

B → A b : replace A with its productions A → B a | a:

A → B a gives B → B a b

A → a gives B → a b

Remove B → A b. Current B productions: B → B a b | a b | b

Remove immediate left recursion from B:

α = "a b", β = "a b", β = "b"

```
B  → a b B'  |  b B'
B' → a b B'  |  ε
```

Final Grammar (no left recursion):

```
A  → B a  |  a
B  → a b B'  |  b B'
B' → a b B'  |  ε
```

9. Left Factoring

9.1 Definition and Purpose

Left factoring is required when two or more productions for the same non-terminal share a common prefix. In such cases, a predictive parser cannot decide which production to apply based on a single lookahead token alone.

9.2 Algorithm

Pattern:

```
A → α β  | α β  | ... | α β  | γ  | γ  | ...
(α is the longest common prefix of any two or more alternatives)
```

Replace with:

```
A  → α A'  | γ  | γ  | ...
A' → β  | β  | ... | β
```

Repeat until no two productions for the same non-terminal share a common prefix.

9.3 Worked Example 1 — Simple Case

Original Grammar:

```
A → a b c  |  a b d  |  e
```

The longest common prefix of "a b c" and "a b d" is "a b".

After Left Factoring:

```
A  → a b A'  |  e
A' → c  |  d
```

9.4 Worked Example 2 — Dangling Else

Original Grammar (if-then-else):

```
S → i E t S e S  |  i E t S  |  a
E → b
```

(i = if, t = then, e = else, a = other statement)

Common prefix: "i E t S" (shared by productions 1 and 2).

After Left Factoring:

```
S  → i E t S S'  |  a
S' → e S  |  ε
E  → b
```

Note: This grammar is still ambiguous (dangling-else problem), so it remains non-LL(1) despite left factoring. Left factoring alone does not guarantee an LL(1) grammar.

9.5 Worked Example 3 — Multi-Level Factoring

Original Grammar:

```
A → a  |  a b  |  a b c
```

Pass 1 — Factor "a":

```
A  → a A'
A' → ε  |  b  |  b c
```

Pass 2 — Factor "b" from A':

```
A  → a A'
A' → ε  |  b A''
A'' → ε  |  c
```

The grammar now has no common prefixes and can be verified for LL(1).

10. Building a Recursive Descent Parser

10.1 Step-by-Step Construction Procedure

Step 1: Remove all left recursion (Section 8).

Step 2: Apply left factoring to eliminate common prefixes (Section 9).

Step 3: Compute FIRST and FOLLOW sets (Sections 5 and 6).

Step 4: Construct and verify the predictive parsing table (Section 7).

Step 5: Write one recursive procedure per non-terminal.

Step 6: Use FIRST/FOLLOW and the parsing table to choose productions inside each procedure.

10.2 Complete Parser — Grammar G1

The following pseudocode implements a complete recursive descent parser for Grammar G1:

```
// ----------------------------------------------------------------
//  Recursive Descent Parser for Grammar G1
//  E  → T E'    E' → + T E' | ε
//  T  → F T'    T' → * F T' | ε
//  F  → ( E ) | id
// ----------------------------------------------------------------

global  lookahead ← first_token()

procedure match(t):
```

```
    if lookahead = t:
        lookahead ← next_token()
    else:
        error("Syntax error: expected '" + t + "', got '" + lookahead + "'")

procedure E():
    // FIRST(T E') = { (, id }
    if lookahead   { (, id }:
        T()
        E_prime()
    else:
        error("Syntax error in E: expected ( or id")

procedure E_prime():
    // E' → + T E'  |  ε
    if lookahead = '+':              // FIRST(+ T E') = { + }
        match('+')
        T()
        E_prime()
    // else ε production: lookahead   FOLLOW(E') = { ), $ } — do nothing

procedure T():
    // FIRST(F T') = { (, id }
    if lookahead   { (, id }:
        F()
        T_prime()
    else:
        error("Syntax error in T: expected ( or id")

procedure T_prime():
    // T' → * F T'  |  ε
    if lookahead = '*':              // FIRST(* F T') = { * }
        match('*')
        F()
        T_prime()
    // else ε production: lookahead   FOLLOW(T') = { +, ), $ } — do nothing

procedure F():
    // F → ( E )  |  id
    if lookahead = '(':              // FIRST(( E )) = { ( }
        match('(')
        E()
        match(')')
    elif lookahead = 'id':           // FIRST(id) = { id }
        match('id')
    else:
        error("Syntax error in F: expected '(' or 'id'")

// ---- Main driver ----
procedure parse():
    E()
    if lookahead ≠ '$':
        error("Input not fully consumed")
    else:
        print("Parsing successful!")
```

10.3 Parsing Trace — Input: "id + id * id"

Step	Call Stack Top	Lookahead	Action / Production
1	parse()	id	Call E()
2	E()	id	E → T E' — call T(), then E'()
3	T()	id	T → F T' — call F(), then T'()
4	F()	id	F → id — match("id"), lookahead ← "+"
5	T'()	+	ε production — '+' ∉ FOLLOW(T'), return
6	E'()	+	E' → + T E' — match("+"), call T(), E'()
7	T()	id	T → F T' — call F(), then T'()
8	F()	id	F → id — match("id"), lookahead ← "*"
9	T'()	*	T' → * F T' — match("*"), call F(), T'()
10	F()	id	F → id — match("id"), lookahead ← "$"
11	T'()	$	ε production — '$' ∉ FOLLOW(T'), return
12	E'()	$	ε production — '$' ∉ FOLLOW(E'), return
13	parse()	$	lookahead = $ — SUCCESS!

11. Lab Exercises

Complete all exercises below. Show every step for full credit. Exercises marked [Bonus] are optional.

11.1 Exercise Set A — FIRST and FOLLOW Sets

Task A1 [10 marks]:

Compute FIRST and FOLLOW sets for all non-terminals in the grammar:

```
S → A B C
A → a A  |  ε
B → b B  |  A C d
C → c C  |  ε
```

Task A2 [10 marks]:

Compute FIRST and FOLLOW sets for:

```
S → a A d  |  b B d  |  ε
A → a  |  c
B → b  |  c
```

Task A3 [10 marks]:

Compute FIRST and FOLLOW sets for:

```
P → Q R S
Q → u Q  |  ε
R → v  |  ε
S → w S  |  Q
```

11.2 Exercise Set B — Left Recursion Removal

Task B1 [10 marks]:

Remove left recursion from:

```
S → S a  |  S b  |  c  |  d
```

Task B2 [15 marks]:

Remove all left recursion (including indirect) from:

```
A → B C  |  a
B → C A  |  b
C → A B  |  c
```

Hint: Fix the ordering of non-terminals carefully and apply the general algorithm.

Task B3 [10 marks]:

Remove left recursion from the arithmetic grammar:

```
E → E + T  |  E - T  |  T
T → T * F  |  T / F  |  F
F → ( E )  |  num
```

11.3 Exercise Set C — Left Factoring

Task C1 [10 marks]:

Apply left factoring to:

```
A → a b c  |  a b d  |  a e f
```

Task C2 [10 marks]:

Apply left factoring to (may require multiple passes):

```
S → a S b S  |  a S  |  a
```

Task C3 [10 marks]:

Apply left factoring to:

```
stmt → if expr then stmt else stmt
      |  if expr then stmt
```

```
          |  other_stmt
```

11.4 Exercise Set D — Full LL(1) Pipeline

Task D1 [15 marks]:

For the grammar below, apply all necessary transformations, then verify LL(1) by constructing the parsing table:

```
    S → ( S ) S  |  ε
```

Task D2 [25 marks]:

Working from the following grammar, complete all steps:

```
    E → E + T  |  T
    T → T * F  |  F
    F → ( E )  |  num
```

- (a) Remove left recursion.
- (b) Compute FIRST and FOLLOW sets.
- (c) Construct the predictive parsing table.
- (d) Verify the grammar is LL(1).
- (e) Write the recursive descent parser in pseudocode.
- (f) Trace the parsing of input: "num + num * (num)"

11.5 Bonus — Automatic Grammar Tool [25 bonus marks]

Implement a program in C, C++, Java, or Python that:

- Reads a CFG from a file or standard input.
- Detects and removes all left recursion (immediate and indirect).
- Applies left factoring iteratively until no common prefixes remain.
- Computes and displays FIRST and FOLLOW sets.
- Generates the LL(1) predictive parsing table.
- Reports whether the input grammar is LL(1) and, if not, identifies the conflicting cells.

Test your tool on:

- Grammar G1 from this manual (expected: LL(1) confirmed).
- The grammar S → i E t S e S | i E t S | a, E → b (expected: not LL(1)).
- At least two additional grammars of your own design.

Submit source code, test outputs for all grammars, and a brief report (max 2 pages) explaining your implementation.

12. References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., Addison-Wesley, 2006. [The "Dragon Book"]

[2] K. Cooper and L. Torczon, Engineering a Compiler, 2nd ed., Morgan Kaufmann, 2011.

[3] T. Mogensen, Introduction to Compiler Design, 2nd ed., Springer, 2017.

[4] D. Grune and C. J. H. Jacobs, Parsing Techniques: A Practical Guide, 2nd ed., Springer, 2008.

[5] J. E. Hopcroft, R. Motwani, and J. D. Ullman, Introduction to Automata Theory, Languages, and Computation, 3rd ed., Addison-Wesley, 2006.